
Course	: Diploma in Electronic Systems (EGDF18) : Diploma in Electronic & Computer Engineering (EGDF20)
Module	: Embedded System Design & Technology (EGE320)
Laboratory No.	: Lab 7
Laboratory Title	: Digital Input
Objective	: To illustrate the concept of digital input. : To demonstrate the use of GPIO peripherals. : To develop application in C using GPIO peripherals.

Contents

1. Push Button Interface Circuit
 - 1.1. Push Button Press Detection
 - 1.2. Implementing App Exit Push Button
 - 1.3. Implementing Home Push Button

1 Push Button Interface Circuit

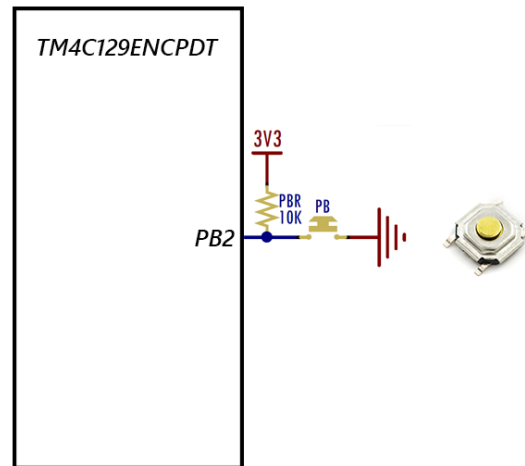


Figure 1a: Push Button Interface Circuit

1.1 Push Button Press Detection

1. The final user interface design and push button press action to be performed is shown in Figure below.

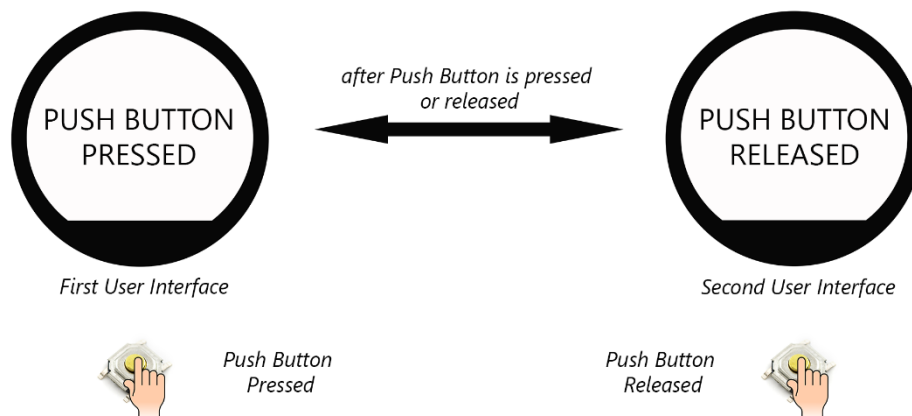


Figure 1.1a: Push Button Press Detection

2. Setup the CCS IDE development environment learned in Lab1.

3. To use an interrupt in the microcontroller to detect the push button press action instead of polling, the following function is needed.

Pointer to the interrupt
handling function

```
void PushButtonIntRegister(void (*pfnIntHandler)(void))
```

```
void PushButtonIntClear(void)
```

4. Complete the code below while entering it into CCS IDE. Build to ensure no errors in the code.

```
#include <stdbool.h>
#include <stdint.h>
#include "clock.h"
#include "glib.h"
#include "st7789_spi.h"
#include "pushbutton.h"

//Include the display driver file
//Include the push button driver file

tContext sContext = {};
uint8_t ui8PressStatus = 0;

void PushButtonIntHandler(void) //Push Button ISR
{
    PushButtonIntClear(); //Clear Interrupt
    if(ui8PressStatus == 0) //Check if ui8PressStatus is 0
        ui8PressStatus = 1; //Set ui8PressStatus to 1
    else
        ui8PressStatus = 0; //Set ui8PressStatus to 0
}

int main(void)
{
    ClockFrequencySet(120000000); //Set processor clock to 120MHz
    ST7789PinInit(120000000); //Initialize LCD Controller pin
    PushButtonPinInit(); //Initialize Push Button pin
    GrContextInit(&sContext, &g_sST7789); //Initialize glib Context

    PushButtonIntRegister( ); //Register Push Button Interrupt

    GrContextBackgroundSet(&sContext, ClrWhite); //Set background color to ClrWhite
    GrContextForegroundSet(&sContext, ClrBlack); //Set foreground color to ClrBlack
    GrContextFontSet(&sContext, g_psFontCmss30); //Set the font type and size
    GrStringDraw(&sContext, "PUSH BUTTON", 11, 30, 80, 1); //Print the "PUSH BUTTON" text
    GrStringDraw(&sContext, "RELEASED", 10, 60, 110, 1); //Print the "RELEASED" text
}
```

```
while(1)
{
    //Check if Push Button is pressed.
    if(ui8PressStatus == )
    {
        //Print the "PRESSED" text
        GrStringDraw(&sContext, "PRESSED ", 10, 60, 110, 1);
    }
    else
    {
        //Print the "RELEASED" text
        GrStringDraw(&sContext, "RELEASED ", 10, 60, 110, 1);
    }
}
```

5. Debug and run the code using F8 key.
6. Press and release on the push button and observe the change on the word displayed on the LCD screen as shown in Figure 1.1a.

1.2 Implementing App Exit Push Button

1. The flow of the user interface design transition after the Push Button is pressed while Torch Light App is running is shown in Figure below.

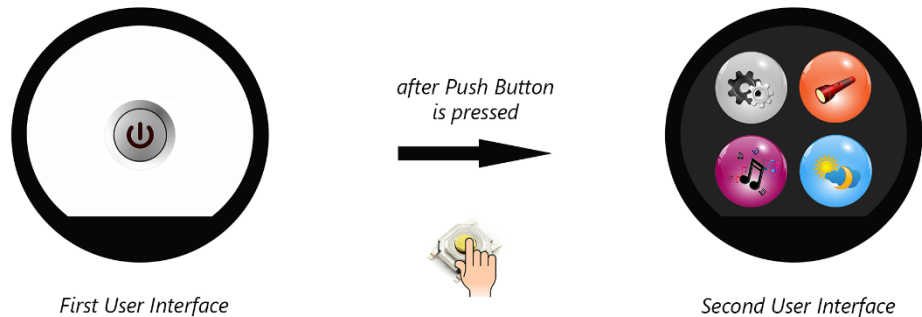


Figure 1.2a: User Interface Design Transition to exit Touch Light App

2. Setup the CCS IDE development environment by importing the working project from Lab6_1.3.
3. Complete the code below by referring to the code in Section 1.1. Build to ensure no errors in the code.

```
#include <stdbool.h>
#include <stdint.h>
#include "clock.h"
#include "glib.h"
#include "st7789_spi.h"
#include "it7258_i2c.h"
#include "rgbled.h"
#include "pushbutton.h"

//Include the display driver file
//Include the touch driver file
//Include the rgbled driver file
//Include the push button driver file

extern const uint8_t g_pui8NypLogo[];
extern const uint8_t g_pui8SettingsIcon[];
extern const uint8_t g_pui8TorchLightIcon[];
extern const uint8_t g_pui8MusicIcon[];
extern const uint8_t g_pui8WeatherIcon[];
extern const uint8_t g_pui8On[];
extern const uint8_t g_pui8Off[];

tContext sContext = {};
tRectangle sRect = {};
uint32_t uiDelay;
uint16_t ui16XPosition[3] = {};
uint16_t ui16YPosition[3] = {};
uint8_t ui8Pressure[3] = {};
bool bTouchStatus = 0;
int8_t i8PageCounter = 0;
bool bPagePrinted = 0;
uint8_t ui8PressStatus = 0;
```

```

void TorchLightApp(void)
{
    bool bLEDStatus = 0;

    //Print Off image
    GrImageDraw(&sContext, g_pui8Off, 80, 80);

    //Continue to loop if ui8PressStatus is 0
    while( )
    {
        if(bTouchStatus == 1)
        {
            IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);

            if((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
            {
                if(((100 < ui16XPosition[0]) && (ui16XPosition[0] < 140)) &&
                    ((100 < ui16YPosition[0]) && (ui16YPosition[0] < 140)))
                {
                    if (bLEDStatus == 0)
                    {
                        RGBLEDOnOff(1, 1, 1);
                        GrImageDraw(&sContext, g_pui8On, 80, 80);
                        bLEDStatus = 1;
                    }
                    else
                    {
                        RGBLEDOnOff(0, 0, 0);
                        GrImageDraw(&sContext, g_pui8Off, 80, 80);
                        bLEDStatus = 0;
                    }
                }
            }

            while((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
            {
                IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);
            }
            bTouchStatus = 0;
        }
    }
    //Reset bPagePrinted to 0
    bPagePrinted = 0;

    //Turn OFF all LEDs
    RGBLEDOnOff(0, 0, 0);

    //Wait until Push Button is released
    while( );
}

void TouchPanelIntHandler(void)
{
    IT7258IntClear();
    bTouchStatus = 1;
}

void PushButtonIntHandler(void)
{
    PushButtonIntClear();
    if(ui8PressStatus == 0)
        ui8PressStatus = 1;
    else
        ui8PressStatus = 0;
}

```

```

int main(void)
{
    ClockFrequencySet(120000000);           //Set processor clock to 120MHz
    ST7789PinInit(120000000);               //Initialize LCD Controller pin
    IT7258PinInit(120000000);               //Initialize Touch Controller pin
    RGBLEDPinInit();                        //Initialize RGBLED pin
    PushButtonPinInit();                    //Initialize Push Button pin

    GrContextInit(&sContext, &g_sST7789);    //Initialize glib Context

    //Register Touch Panel Interrupt Handler
    IT7258IntRegister(TouchPanelIntHandler);

    //Register Push Button Interrupt
    PushButtonIntRegister(PushButtonIntHandler);

    //Code to display NYP logo, static date and time and unlock button
    GrImageDraw(&sContext, g_pui8NypLogo, 20, 90);
    for(uiDelay=0; uiDelay<7000000; uiDelay++);
    GrContextForegroundSet(&sContext, ClrWhite);
    GrCircleFill(&sContext, 120, 120, 121);
    GrContextForegroundSet(&sContext, ClrBlack);
    GrContextFontSet(&sContext, g_psFontCmss48);
    GrContextBackgroundSet(&sContext, ClrWhite);
    GrContextForegroundSet(&sContext, ClrBlack);
    GrStringDraw(&sContext, "09:55:34", 8, 50, 60, 0);
    GrContextFontSet(&sContext, g_psFontCmss28);
    GrStringDraw(&sContext, "01:04:18", 8, 80, 110, 0);
    sRect.i16XMin = 60;
    sRect.i16YMin = 150;
    sRect.i16XMax = 180;
    sRect.i16YMax = 190;
    GrRectFill(&sContext, &sRect);
    GrLineDraw(&sContext, 58, 148, 182, 148);
    GrLineDraw(&sContext, 58, 148, 58, 192);
    GrLineDraw(&sContext, 58, 192, 182, 192);
    GrLineDraw(&sContext, 182, 148, 182, 192);
    GrContextForegroundSet(&sContext, ClrWhite);
    GrContextFontSet(&sContext, g_psFontCmss14);
    GrStringDraw(&sContext, "PRESS TO UNLOCK", 15, 65, 165, 0);

    while(1)
    {
        if(i8PageCounter == 0)
        {
            if(bTouchStatus == 1)
            {
                //Code to enable touch for UNLOCK button
                IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);
                if((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
                {
                    if(((60 < ui16XPosition[0]) && (ui16XPosition[0] < 180)) &&
                       ((50 < ui16YPosition[0]) && (ui16YPosition[0] < 90)))
                    {
                        i8PageCounter = 1;
                    }
                }
                while((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
                {
                    IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);
                }
                bTouchStatus = 0;
            }
        }
    }
}

```

```

if(i8PageCounter == 1)
{
    if(bPagePrinted == 0)
    {
        //Code to display the 4 App Icons
        GrContextForegroundSet(&sContext, ClrBlack);
        GrCircleFill(&sContext, 120, 120, 121);
        GrImageDraw(&sContext, g_pui8SettingsIcon, 35, 25);
        GrImageDraw(&sContext, g_pui8TorchLightIcon, 125, 25);
        GrImageDraw(&sContext, g_pui8MusicIcon, 35, 115);
        GrImageDraw(&sContext, g_pui8WeatherIcon, 125, 115);
        bPagePrinted = 1;
    }

    if(bTouchStatus == 1)
    {
        //Code to detect the touch points for the 4 App Icons
        GrContextForegroundSet(&sContext, ClrWhite);
        IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);

        if((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
        {
            if(((125 < ui16XPosition[0]) && (ui16XPosition[0] < 205)) &&
                ((115 < ui16YPosition[0]) && (ui16YPosition[0] < 195)))
            {
                GrCircleFill(&sContext, 120, 120, 121);
            }
            if(((35 < ui16XPosition[0]) && (ui16XPosition[0] < 115)) &&
                ((115 < ui16YPosition[0]) && (ui16YPosition[0] < 195)))
            {
                GrCircleFill(&sContext, 120, 120, 121);
                TorchLightApp();
            }
            if(((125 < ui16XPosition[0]) && (ui16XPosition[0] < 205)) &&
                ((25 < ui16YPosition[0]) && (ui16YPosition[0] < 105)))
            {
                GrCircleFill(&sContext, 120, 120, 121);
            }
            if(((35 < ui16XPosition[0]) && (ui16XPosition[0] < 115)) &&
                ((25 < ui16YPosition[0]) && (ui16YPosition[0] < 105)))
            {
                GrCircleFill(&sContext, 120, 120, 121);
            }
        }

        while((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
        {
            IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);
        }

        bTouchStatus = 0;
    }
}
}
}

```

4. Debug and run the code using F8 key.
5. Touch on the Unlock button displayed on the LCD screen to access to the user interface with four App Icons. Continue to touch on Torch Light App icon to access the application. Press on the Push Button to exit from the App as shown in Figure 1.2a

1.3 Implementing Home Push Button

1. The flow of the user interface design transition after the Push Button is pressed from the App User Interface is shown in Figure below.

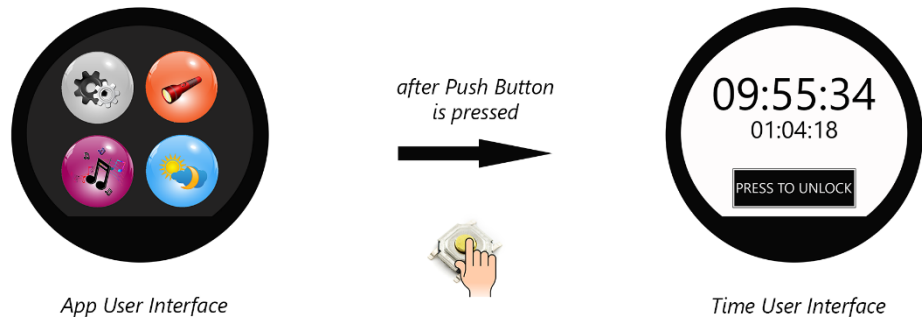


Figure 1.3a: User Interface Transition by Pressing Home Push Button

2. Setup the CCS IDE development environment by importing the working project from section 1.2.
3. Complete the code below. Build to ensure no errors in the code.

```
#include <stdbool.h>
#include <stdint.h>
#include "clock.h"
#include "gplib.h"
#include "st7789_spi.h"
#include "it7258_i2c.h"
#include "rgbled.h"
#include "pushbutton.h"

//Include the display driver file
//Include the touch driver file
//Include the rgbled driver file
//Include the push button driver file

extern const uint8_t g_pui8NypLogo[];
extern const uint8_t g_pui8SettingsIcon[];
extern const uint8_t g_pui8TorchLightIcon[];
extern const uint8_t g_pui8MusicIcon[];
extern const uint8_t g_pui8WeatherIcon[];
extern const uint8_t g_pui8On[];
extern const uint8_t g_pui8Off[];

tContext sContext = {};
tRectangle sRect = {};
uint32_t uiDelay;
uint16_t ui16XPosition[3] = {};
uint16_t ui16YPosition[3] = {};
uint8_t ui8Pressure[3] = {};
bool bTouchStatus = 0;
int8_t i8PageCounter = 0;
bool bPagePrinted = 0;
uint8_t ui8PressStatus = 0;
bool bHomePagePrinted = 0;

void TorchLightApp(void)
{
    //Insert the TorchLightApp function code from (Exercise 1.2)
}
```

```

void TouchPanelIntHandler(void)
{
    IT7258IntClear();
    bTouchStatus = 1;
}

void PushButtonIntHandler(void)
{
    PushButtonIntClear();
    if(ui8PressStatus == 0)
        ui8PressStatus = 1;
    else
        ui8PressStatus = 0;
}

int main(void)
{
    ClockFrequencySet(120000000);           //Set processor clock to 120MHz
    ST7789PinInit(120000000);               //Initialize LCD Controller pin
    IT7258PinInit(120000000);               //Initialize Touch Controller pin
    RGBLEDPinInit();                       //Initialize RGBLED pin
    PushButtonPinInit();                   //Initialize Push Button pin

    GrContextInit(&sContext, &g_sST7789);   //Initialize glib Context

    //Register Touch Panel and Push Button Interrupt Handler
    IT7258IntRegister(TouchPanelIntHandler);
    PushButtonIntRegister(PushButtonIntHandler);

    //Code to display NYP logo, static date and time and unlock button
    GrImageDraw(&sContext, g_pui8NypLogo, 20, 90);
    for(uiDelay=0; uiDelay<7000000; uiDelay++);
    GrContextForegroundSet(&sContext, ClrWhite);
    GrCircleFill(&sContext, 120, 120, 121);
    GrContextForegroundSet(&sContext, ClrBlack);
    GrContextFontSet(&sContext, g_psFontCmss48);
    GrContextBackgroundSet(&sContext, ClrWhite);
    GrContextForegroundSet(&sContext, ClrBlack);
    GrStringDraw(&sContext, "09:55:34", 8, 50, 60, 0);
    GrContextFontSet(&sContext, g_psFontCmss28);
    GrStringDraw(&sContext, "01:04:18", 8, 80, 110, 0);
    sRect.i16XMin = 60;
    sRect.i16YMin = 150;
    sRect.i16XMax = 180;
    sRect.i16YMax = 190;
    GrRectFill(&sContext, &sRect);
    GrLineDraw(&sContext, 58, 148, 182, 148);
    GrLineDraw(&sContext, 58, 148, 58, 192);
    GrLineDraw(&sContext, 58, 192, 182, 192);
    GrLineDraw(&sContext, 182, 148, 182, 192);
    GrContextForegroundSet(&sContext, ClrWhite);
    GrContextFontSet(&sContext, g_psFontCmss14);
    GrStringDraw(&sContext, "PRESS TO UNLOCK", 15, 65, 165, 0);

    //Insert code to set bHomePagePrinted to 1

    while(1)
    {
        if(i8PageCounter == 0)
        {
            if(bHomePagePrinted == 0)
            {
                //Insert Code to clear screen, display static date and time
                //and unlock button

                //Insert code to set bHomePagePrinted to 1
            }
        }
    }
}

```

```

    if(bTouchStatus == 1)
    {
        //Code to enable touch for UNLOCK button
        IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);
        if((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
        {
            if(((60 < ui16XPosition[0]) && (ui16XPosition[0] < 180)) &&
                ((50 < ui16YPosition[0]) && (ui16YPosition[0] < 90)))
            {
                i8PageCounter = 1;
            }
        }
        while((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
        {
            IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);
        }
        bTouchStatus = 0;
    }
}

if(i8PageCounter == 1)
{
    if(bPagePrinted == 0)
    {
        //Code to display the 4 App Icons
        GrContextForegroundSet(&sContext, ClrBlack);
        GrCircleFill(&sContext, 120, 120, 121);
        GrImageDraw(&sContext, g_pui8SettingsIcon, 35, 25);
        GrImageDraw(&sContext, g_pui8TorchLightIcon, 125, 25);
        GrImageDraw(&sContext, g_pui8MusicIcon, 35, 115);
        GrImageDraw(&sContext, g_pui8WeatherIcon, 125, 115);
        bPagePrinted = 1;
    }

    if(bTouchStatus == 1)
    {
        //Code to detect the touch points for the 4 App Icons
        GrContextForegroundSet(&sContext, ClrWhite);
        IT7258GetTouchPointData(ui16XPosition, ui16YPosition, ui8Pressure);

        if((ui16XPosition[0] != 0) && (ui16YPosition[0] != 0))
        {
            if(((125 < ui16XPosition[0]) && (ui16XPosition[0] < 205)) &&
                ((115 < ui16YPosition[0]) && (ui16YPosition[0] < 195)))
            {
                GrCircleFill(&sContext, 120, 120, 121);
            }
            if(((35 < ui16XPosition[0]) && (ui16XPosition[0] < 115)) &&
                ((115 < ui16YPosition[0]) && (ui16YPosition[0] < 195)))
            {
                GrCircleFill(&sContext, 120, 120, 121);
                TorchLightApp();
            }
            if(((125 < ui16XPosition[0]) && (ui16XPosition[0] < 205)) &&
                ((25 < ui16YPosition[0]) && (ui16YPosition[0] < 105)))
            {
                GrCircleFill(&sContext, 120, 120, 121);
            }
            if(((35 < ui16XPosition[0]) && (ui16XPosition[0] < 115)) &&
                ((25 < ui16YPosition[0]) && (ui16YPosition[0] < 105)))
            {
                GrCircleFill(&sContext, 120, 120, 121);
            }
        }
    }
}

```

